

+ - PROJECT REPORT

AWS Compute Optimizer Driven Rightsizing Programme

A step-by-step account of building a rightsizing demo on AWS: from the problem statement to the first CloudWatch metrics

Team Members:

2400032408 — Prem Prakash
2400031777 — Tentu Divya Amrutha
2400031637 — Chimakurthi Lakshmi Janani
2400031630 — Samarth V Ratnam

Date: 4 October 2026

Region: US East (N. Virginia), us-east-1

Services: Amazon EC2, Amazon CloudWatch, AWS Compute Optimizer

Contents

1. The Problem Statement	3
1.1 Breaking the problem down	3
1.2 Objectives	3
2. The Plan and Architecture	4
3. The Journey, Step by Step	4
Step 1: Starting at the Compute Optimizer dashboard	4
Step 2: Checking what was running in EC2	5
Step 3: Launching the demo instance, naming it	6
Step 4: Choosing the instance type and key pair	6
Step 5: Network and security settings	7
Step 6: Launch succeeded	8
Step 7: Confirming the instance is running	9
Step 8: Tagging the instance	9
Step 9: Watching the metrics appear in CloudWatch	11
Step 10: Returning to CloudWatch, the readings arrive	11
4. Errors and Challenges Faced	12
5. What We Learned	13
6. Current Status and Next Steps	13
7. Conclusion	14

1. The Problem Statement

Cloud bills rarely grow because of one big mistake. They grow quietly, one oversized server at a time. A team launches a large instance "just to be safe", the workload turns out to need a fraction of that capacity, and nobody goes back to check. The instance keeps running, and keeps costing money, month after month.

The task given to us was titled the AWS Compute Optimizer Driven Rightsizing Programme. In plain words: use data, not guesswork, to decide whether the compute resources we run are the right size, and build a working demonstration of how that is done on AWS.

1.1 Breaking the problem down

Before touching the console, we split the problem statement into its working parts. Each part later became a stage of the project.

Part of the problem	What it means	How we addressed it
Rightsizing	Matching an instance's size (vCPU, memory, network) to what its workload actually uses.	Launch a small instance whose utilization can be measured and judged.
Driven by data	Decisions must come from observed utilization, not assumptions.	Rely on Amazon CloudWatch metrics (CPU, network, EBS) as the evidence.
Compute Optimizer	The AWS service that analyses utilization history and recommends better instance types.	Enable and use it as the analysis engine.
Programme	A repeatable process rather than a one-off fix.	Tag resources consistently (Project, Environment, Purpose) and document each step so it can be repeated.

1.2 Objectives

- Create a clean, labelled demo workload (an EC2 instance) in a single region.
- Make sure its utilization data flows into Amazon CloudWatch.
- Allow AWS Compute Optimizer to analyse that data and produce rightsizing findings.
- Document every step, including the problems encountered, as a report that can be reused.

2. The Plan and Architecture

The design is deliberately simple. A single EC2 instance produces utilization metrics. CloudWatch collects and stores those metrics. Compute Optimizer reads that history and compares it with what the instance could have been, then recommends a better fit if one exists.

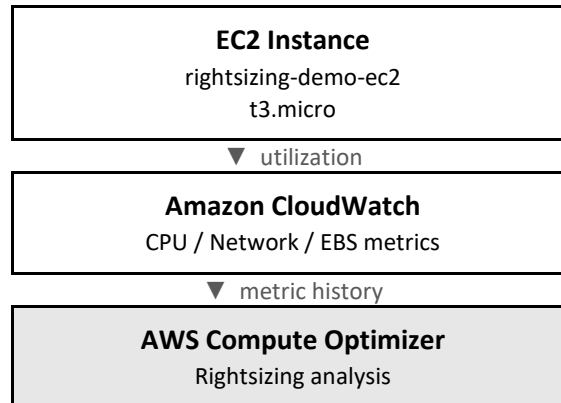


Figure A: Data flow of the rightsizing demo

One design decision is worth stating early. We chose a t3.micro instance (2 vCPU, 1 GiB memory) because it is cheap to run, belongs to a current-generation family, and is small enough that the demo costs almost nothing while it collects data.

3. The Journey, Step by Step

Step 1: Starting at the Compute Optimizer dashboard

We began where the story ends, at the AWS Compute Optimizer dashboard in us-east-1. The left-hand menu already lists everything the service can analyse: EC2 instances, Auto Scaling groups, EBS volumes, Lambda functions, Aurora and RDS databases, and ECS services on Fargate.

The dashboard, however, was empty. Both the "Estimated savings opportunities" and the "Estimated performance improvement opportunities" panels read "Data not available", and a banner explained that Compute Optimizer may take up to 12 hours to deliver findings. This was our first lesson: Compute Optimizer cannot recommend anything until there is a workload to look at, and the data takes time to arrive.

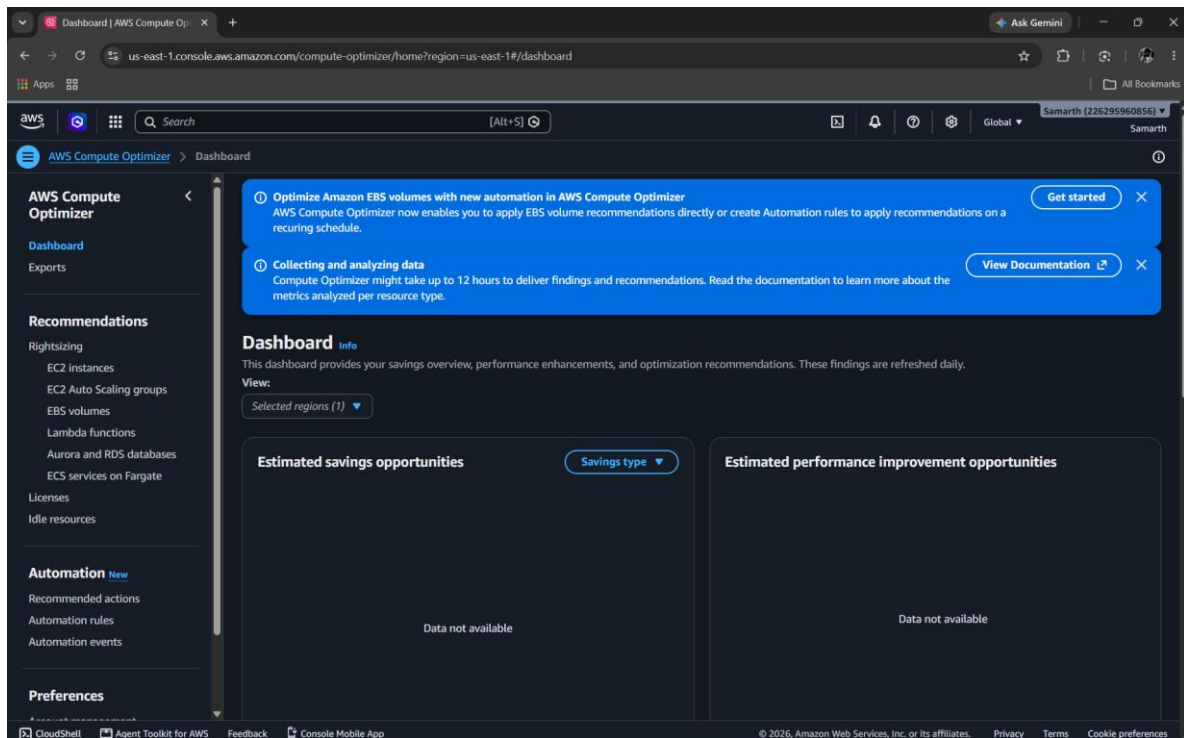


Figure 1: AWS Compute Optimizer dashboard showing "Data not available" before any workload existed

Step 2: Checking what was running in EC2

To understand why the dashboard was empty, we opened the EC2 dashboard for the same region. The Resources panel confirmed it: zero running instances, zero volumes, zero snapshots. The account only held a few key pairs and security groups. The EC2 cost panel also showed no data. In short, there was nothing for Compute Optimizer to analyse.

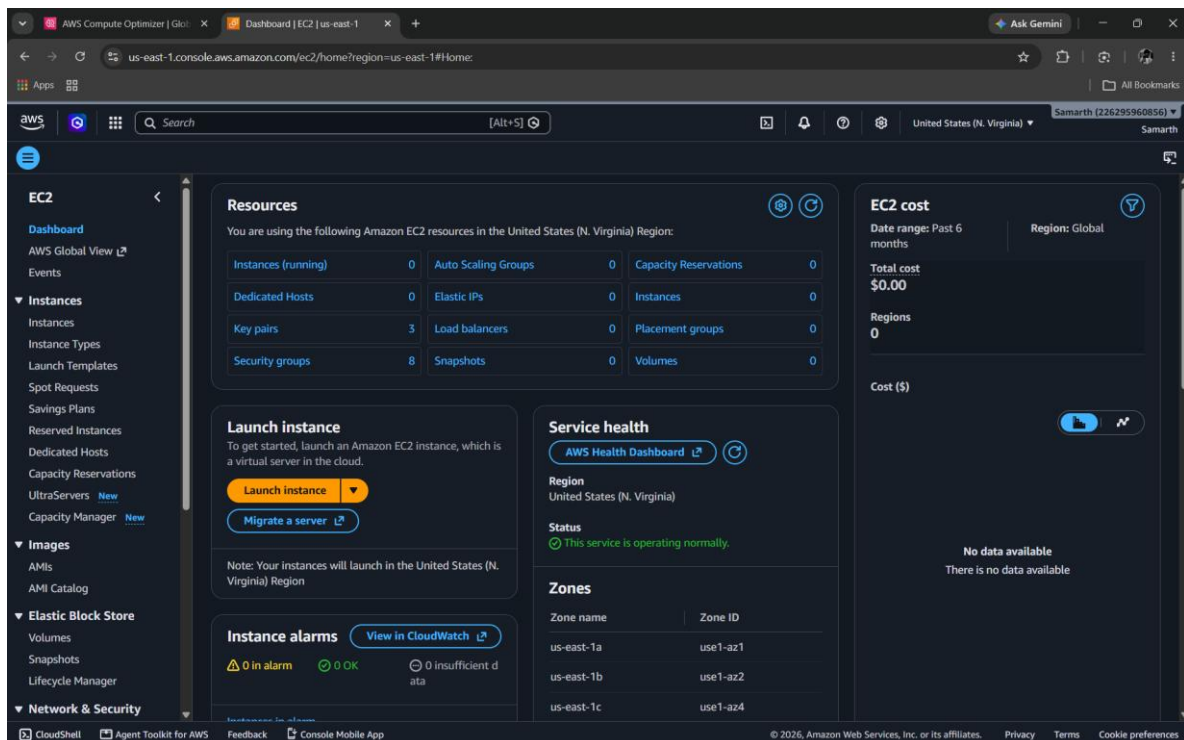


Figure 2: EC2 dashboard in N. Virginia showing no running instances

Step 3: Launching the demo instance, naming it

We started the Launch Instance wizard and gave the server a clear name, rightsizing-demo-ec2, so that it could be recognised instantly in CloudWatch and Compute Optimizer. For the operating system we kept the Amazon Linux quick-start image (Amazon Linux 2023), which is free of licence charges and lightweight.

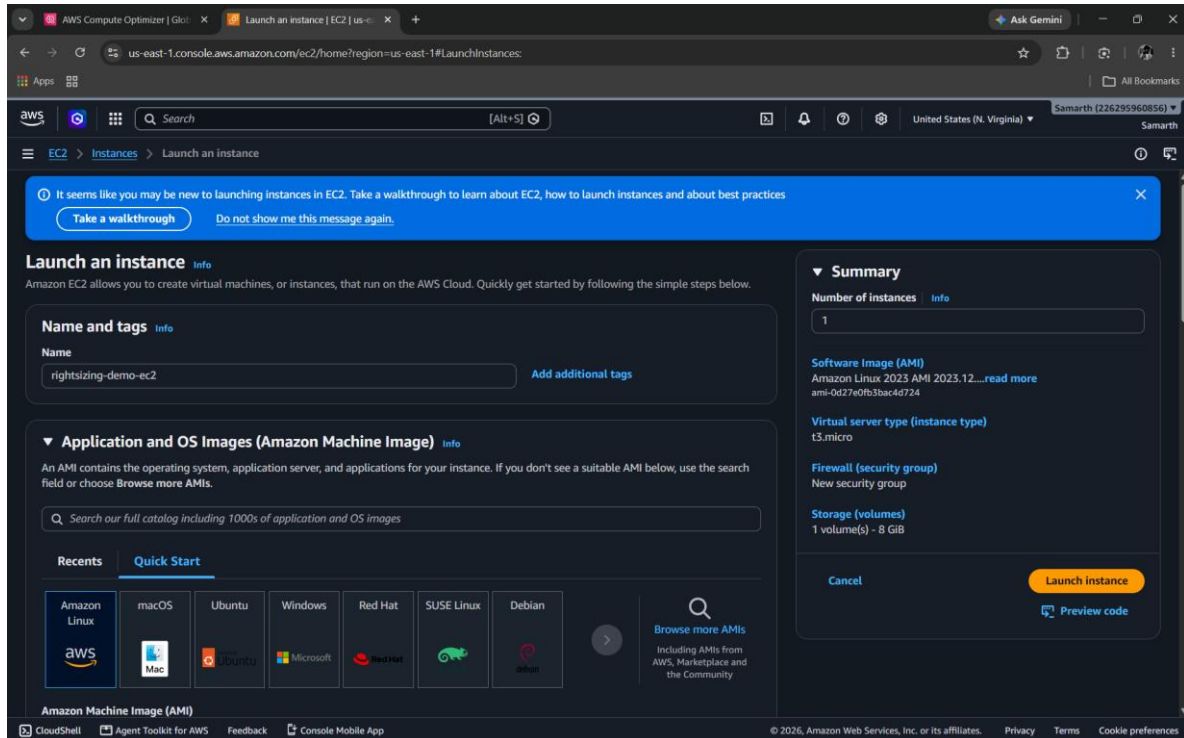


Figure 3: Launch wizard: instance name and Amazon Linux 2023 image

Step 4: Choosing the instance type and key pair

Next came the most important choice of the project: the instance type. We selected t3.micro (2 vCPU, 1 GiB memory), whose on-demand Linux price is shown in the wizard as 0.0104 USD per hour. We also selected an existing key pair, rightsizing-demo-key, which is what allows secure SSH login later.

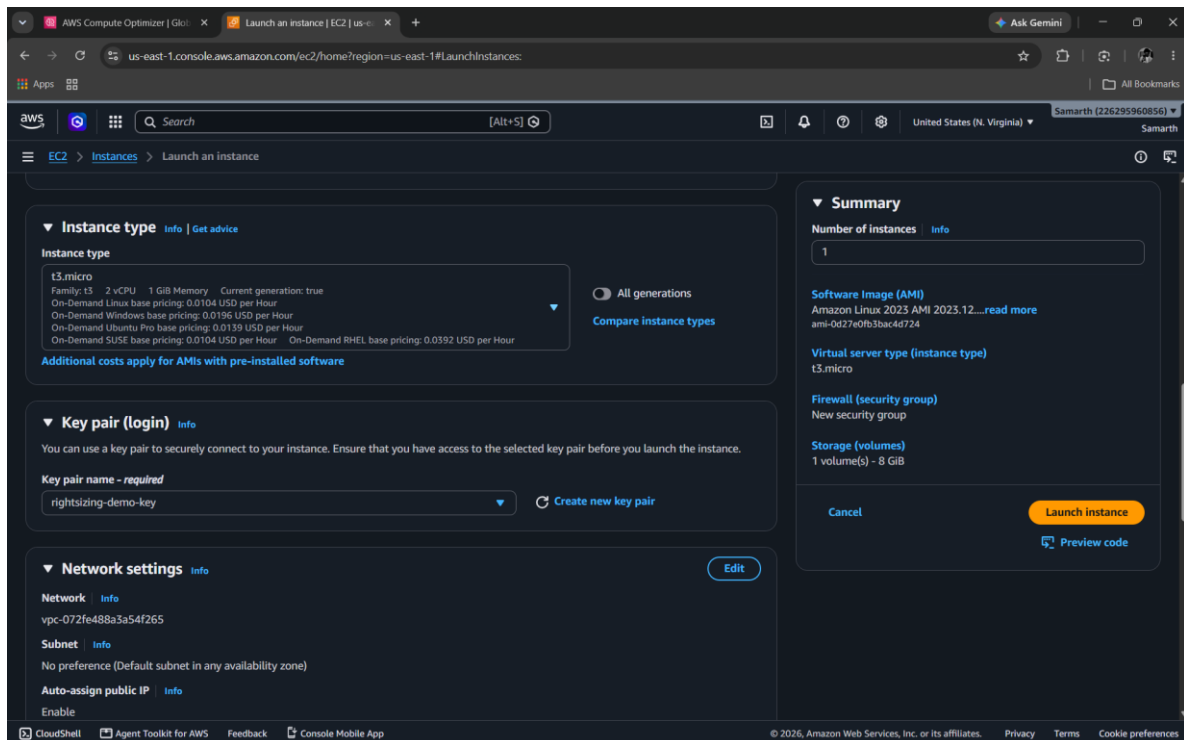


Figure 4: Instance type (t3.micro) and key pair selection

Step 5: Network and security settings

We kept the default VPC and let AWS pick a default subnet, with a public IP enabled. For the firewall we created a new security group and allowed SSH only from our own IP address, using a /32 rule. HTTP and HTTPS were left unticked because this demo does not host a website. Opening only what is needed is a small habit, but it keeps a demo server from becoming a security risk.

Storage was left at the default: one 8 GiB gp3 root volume with 3000 IOPS, not encrypted.

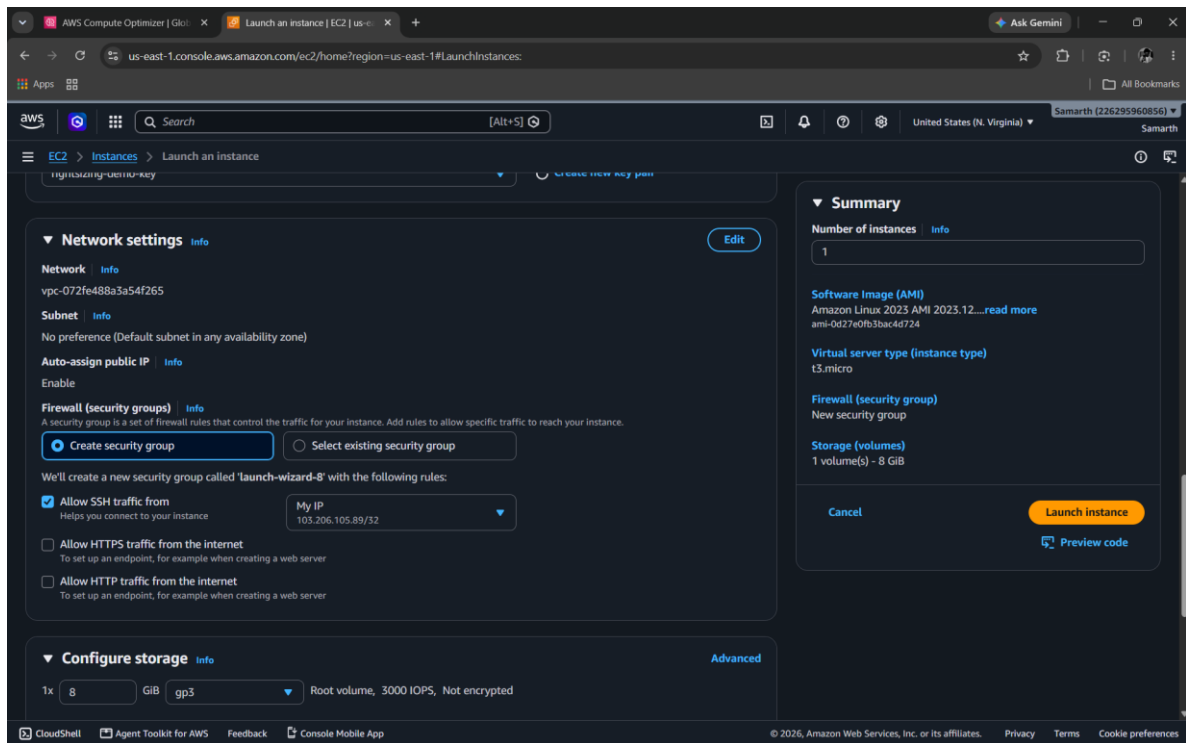


Figure 5: Network settings, security group rules and root storage

Step 6: Launch succeeded

With the settings reviewed, we pressed Launch instance. AWS confirmed that the launch of instance `i-0f37ca7c92798404c` had been initiated successfully, and offered a list of next steps such as billing alerts, connecting to the instance, and CloudWatch alarms.

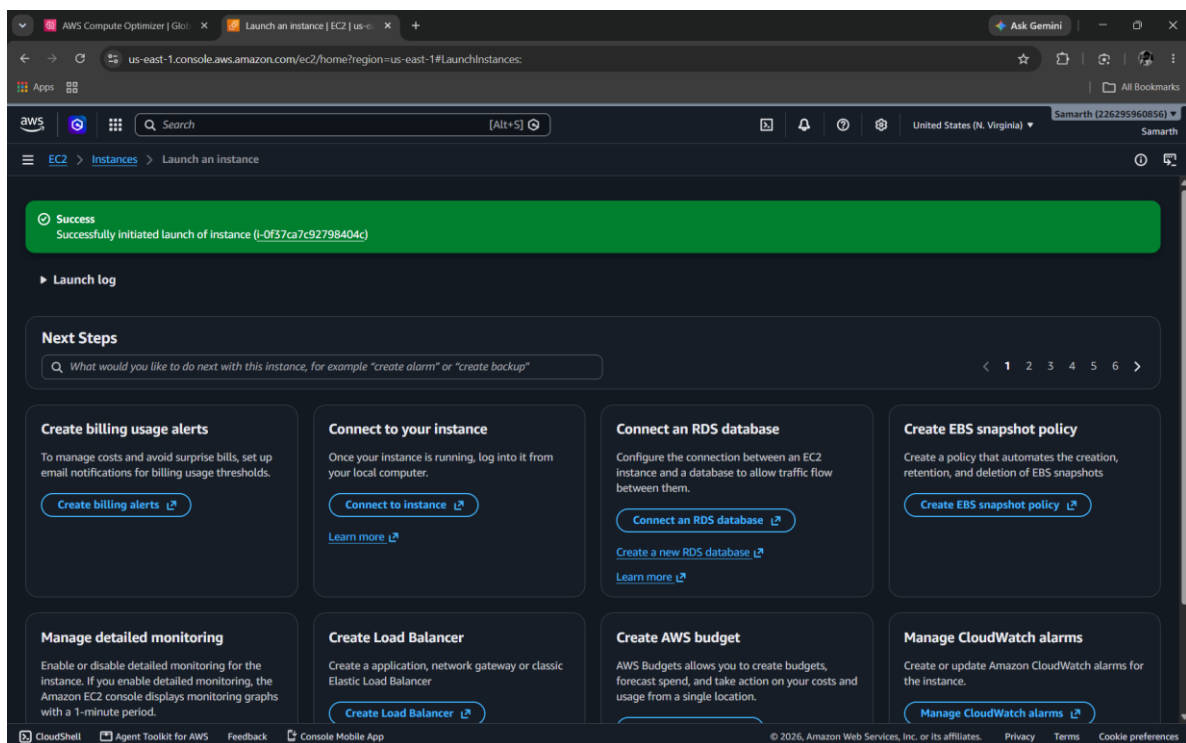


Figure 6: Launch success message for instance `i-0f37ca7c92798404c`

Step 7: Confirming the instance is running

On the Instances page the new server appeared as Running, type t3.micro, in availability zone us-east-1c. The status check, though, still said "Initializing". This is normal for a brand-new instance and is explained in the next section.

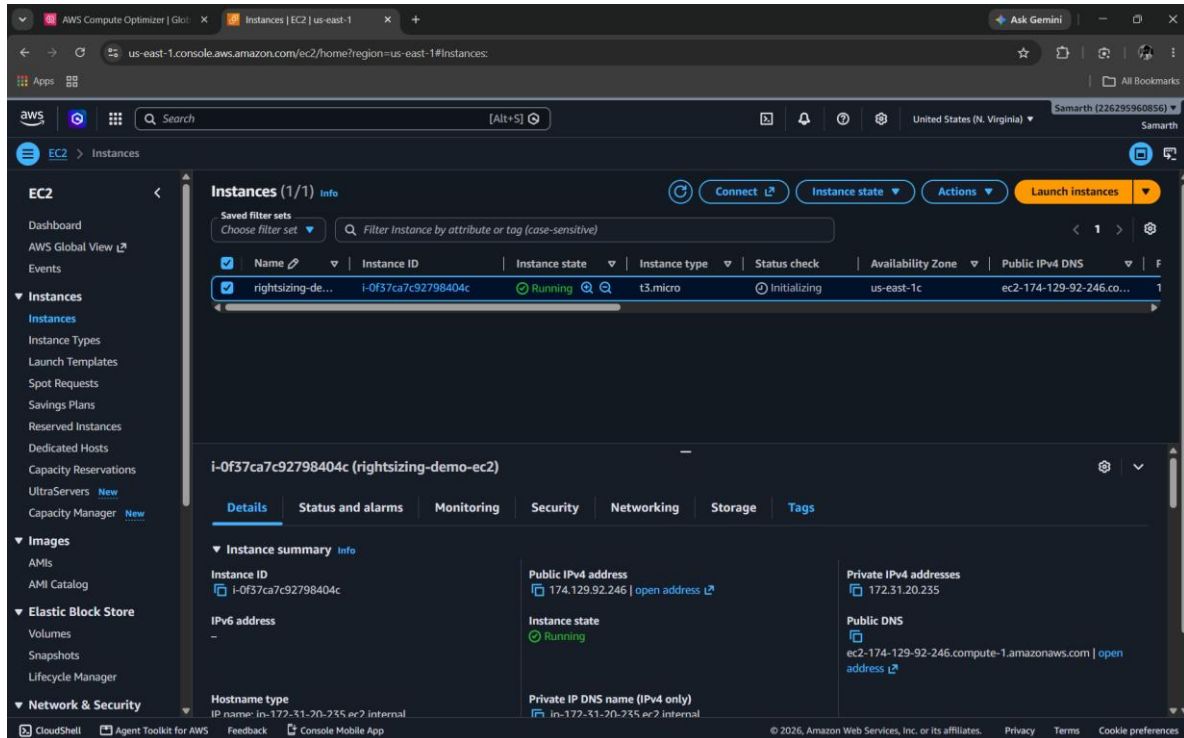


Figure 7: Instances page: rightsizing-demo-ec2 running, status checks initializing

Step 8: Tagging the instance

A rightsizing programme needs to be repeatable and auditable, so we tagged the instance straight away through Manage tags. Besides the Name tag, we added three more:

- **Project:** Compute-Optimizer-Rightsizing
- **Environment:** Demo
- **Purpose:** Rightsizing Analysis

These tags let us filter costs and resources by project later, and make it obvious to anyone else in the account why this server exists.

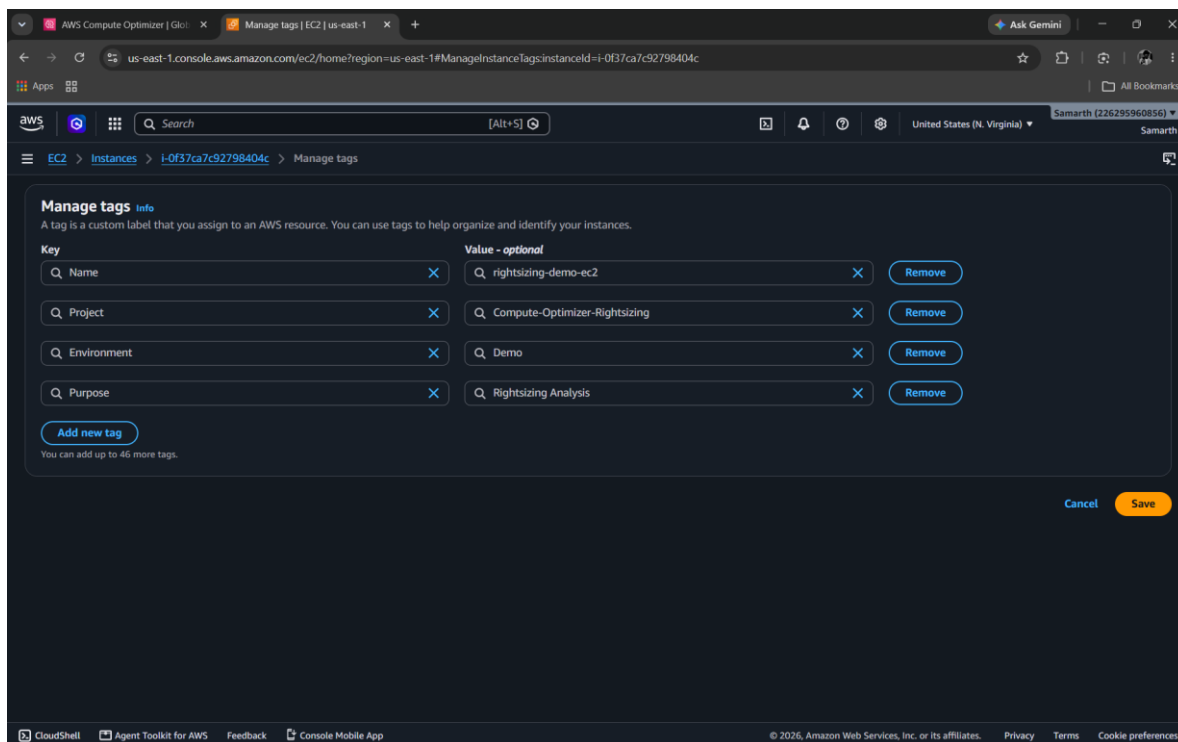


Figure 8: Manage tags screen with Name, Project, Environment and Purpose tags

AWS confirmed the change with "Request to manage tags has succeeded".

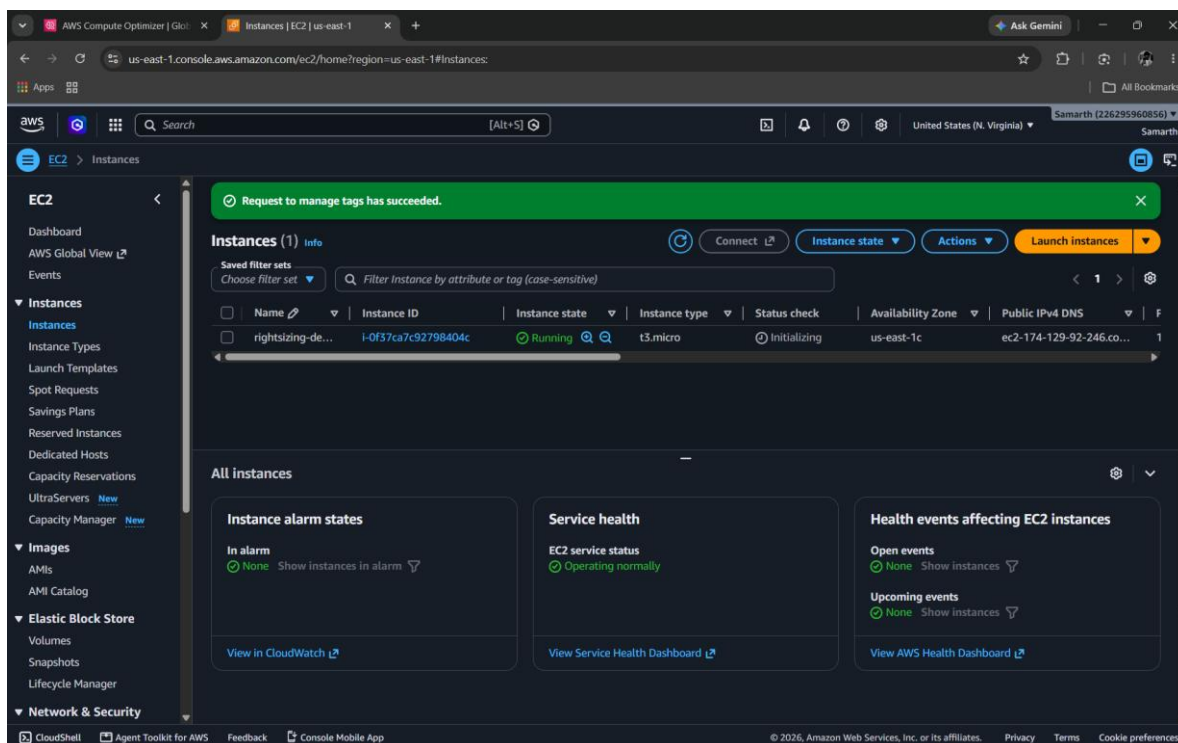


Figure 9: Tag update confirmed on the Instances page

Step 9: Watching the metrics appear in CloudWatch

Finally we opened Amazon CloudWatch and filtered the metrics by our instance ID. CloudWatch had already started recording 22 metrics for the instance. We graphed three of the most relevant ones for rightsizing: CPUUtilization, NetworkIn and NetworkOut.

The first data points were tiny. At 00:10 UTC on 4 October 2026 the CPU utilization was about 0.07 percent, and network traffic was a few hundred bytes. That is exactly what a freshly booted, idle server looks like, and it is the raw evidence Compute Optimizer will later weigh up.

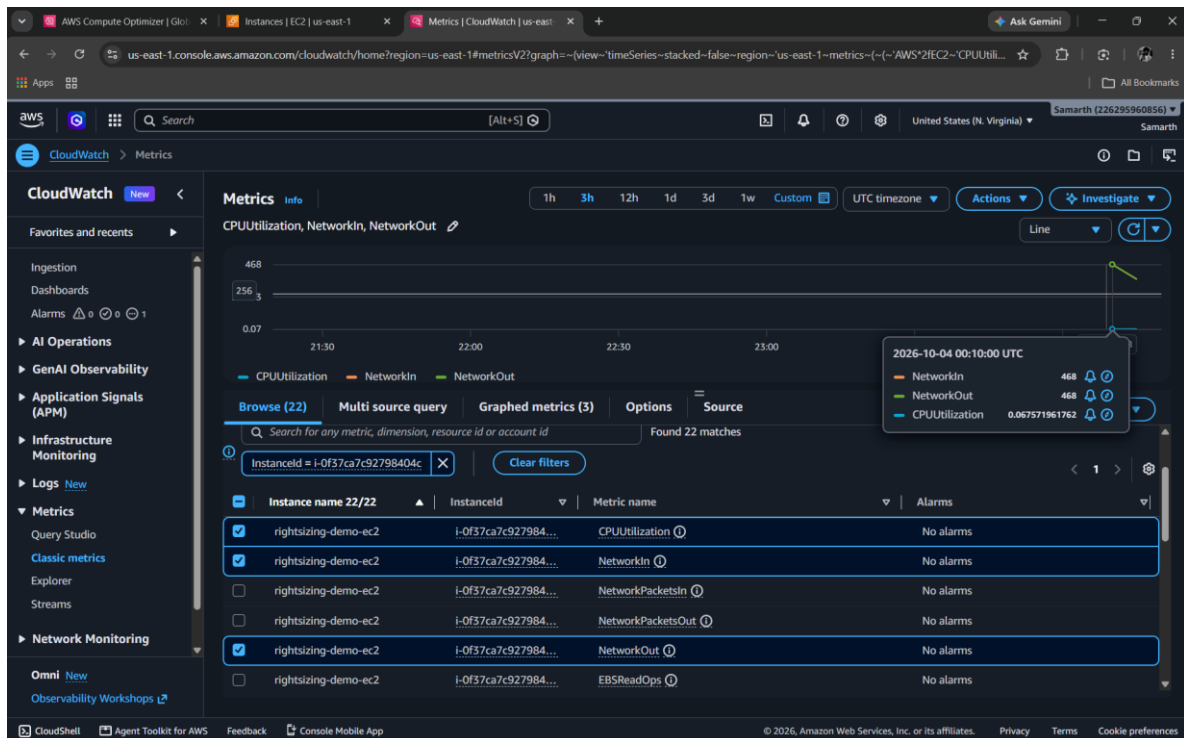


Figure 10: CloudWatch graph of CPUUtilization, NetworkIn and NetworkOut for the demo instance

Step 10: Returning to CloudWatch, the readings arrive

A little over three hours later we went back to the same CloudWatch graph, and this time there was a real series of readings to look at. The picture is now a proper history rather than a single point.

The graph shows a regular pattern. Network traffic sits near zero most of the time and then jumps to a sharp peak roughly every 30 minutes, with NetworkIn and NetworkOut rising and falling together. CPU utilization stays flat along the bottom of the chart through the whole window.

Reading (01:35 UTC, 4 Oct 2026)	Value	Unit
NetworkIn	1,553.6	Bytes per 5-minute period
NetworkOut	1,146.2	Bytes per 5-minute period
CPUUtilization	0.173	Percent

These numbers are the heart of the rightsizing story. A t3.micro with only about 0.17 percent CPU use at its busiest sampled moment is barely working. Compute Optimizer will weigh this long, quiet history when it decides whether the instance is over-provisioned.

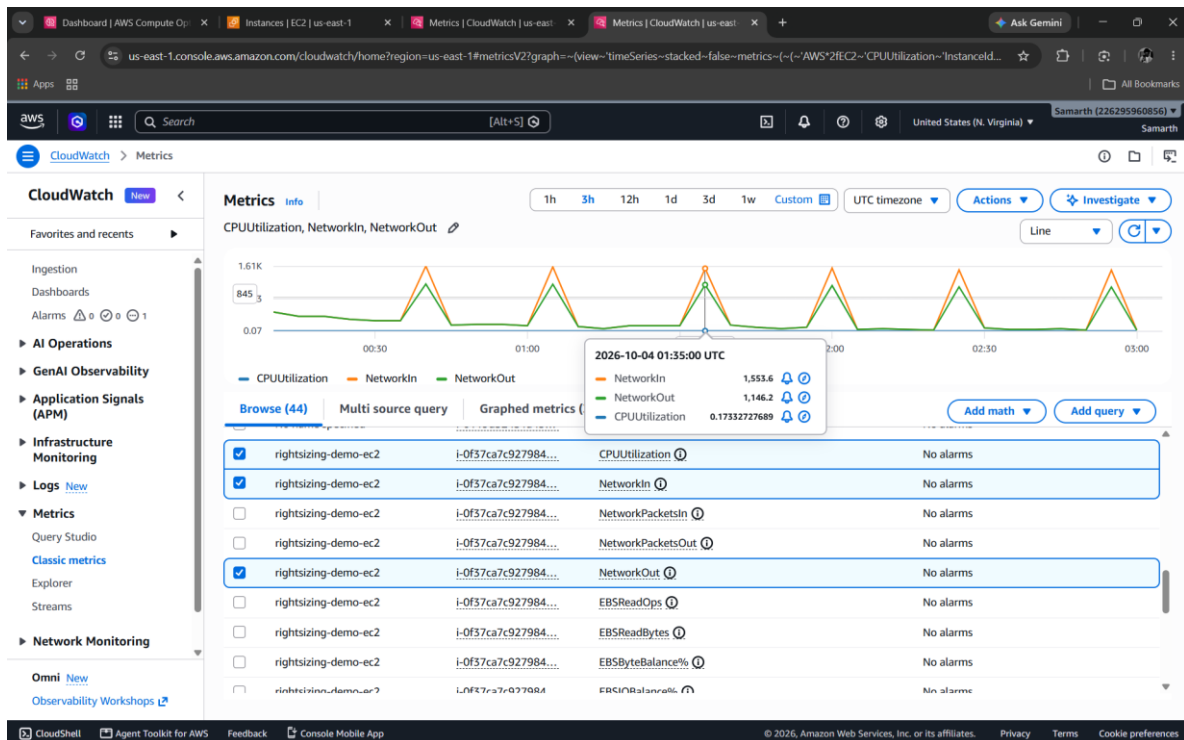


Figure 11: CloudWatch readings over about three hours, with values at 01:35 UTC

4. Errors and Challenges Faced

No cloud project runs perfectly the first time. These are the points where we got stuck or confused, what caused each one, and how it was handled.

Issue	Where we saw it	Cause	Resolution
"Data not available" on the dashboard	Compute Optimizer dashboard (Figure 1)	No workload existed yet, and findings take up to 12 hours to appear even after one does.	Launched the demo instance and allowed time for data to accumulate.
Empty EC2 account	EC2 dashboard (Figure 2)	No instances, volumes or snapshots in us-east-1.	Created rightsizing-demo-ec2 through the Launch Instance wizard.
Status check "Initializing"	Instances page (Figures 7 and 9)	AWS runs system and instance checks after boot; they need a few minutes to pass.	Waited and refreshed. Not a fault.
Tags not set at launch	Launch wizard (Figure 3) only offered the Name tag	Extra tags were not added during the wizard.	Added Project, Environment and Purpose afterwards with Manage tags (Figure 8).

Issue	Where we saw it	Cause	Resolution
Almost no activity in the metrics	CloudWatch graphs (Figures 10 and 11)	The instance is idle, so CPU stays between about 0.07 and 0.17 percent and network traffic is minimal.	Treated as expected for a new demo server; see the next steps for adding load.

5. What We Learned

- **Compute Optimizer needs history, not a snapshot.** It analyses utilization over time, so a newly launched instance cannot be assessed on day one. AWS documentation states that an instance needs roughly 30 hours of metric data before it is evaluated.
- **The readings show a lightly used server.** Over the first few hours CPU never rose above about 0.17 percent, while network traffic showed small peaks about every 30 minutes. That is a strong hint the instance is larger than its workload needs.
- **Naming and tagging pay off immediately.** Because the instance carried a clear name and tags, it was easy to find in CloudWatch and will be easy to identify in the recommendations.
- **Right-sizing starts with a sensible first guess.** A t3.micro is already small. If Compute Optimizer still judges it over-provisioned, that tells us a lot about how generously servers are normally sized.
- **Small security habits matter.** Restricting SSH to a single IP and leaving web ports closed kept the demo safe at no extra cost.

6. Current Status and Next Steps

At the time of writing, the demo environment is in place and measurable: the instance is running, tagged, and reporting to CloudWatch. What remains is the analysis half of the programme.

- Confirm that the account is opted in to AWS Compute Optimizer, and check that the dashboard region matches us-east-1.
- Leave the instance running so that enough utilization history builds up for Compute Optimizer to evaluate it.
- Optionally generate a controlled workload (for example a CPU stress test over SSH) so the metrics show more than idle behaviour.
- Return to Compute Optimizer, open Rightsizing, then EC2 instances, and read the finding: Under-provisioned, Optimized or Over-provisioned, along with the recommended instance types and projected savings.
- Record the recommendation and savings in this report, then stop or terminate the instance when the demo is finished to avoid unnecessary charges.

7. Conclusion

We started with a broad idea, using data to rightsize cloud compute, and turned it into a concrete, working pipeline: a labelled EC2 instance feeding CloudWatch, with Compute Optimizer ready to analyse it. Along the way we met the usual first-time hurdles: an empty dashboard, a status check that was still initializing, and metrics that were quiet because the server was idle. Each had a clear and ordinary explanation.

The result is a repeatable foundation. The same pattern of tagged resources, CloudWatch metrics and Compute Optimizer findings can be applied to larger fleets, where the savings from choosing the right instance size become real.